

Class-3

Topics: the missing-and-repeated number problem (five solutions, ending with pure math), linked lists, the slow & fast pointer technique (loop detection + finding the start of the loop, with proof), "implement a stack" as an interview question, and the monotonic stack pattern.

Before we start — how to study this course. Repetition beats novelty. Re-solving today's problems *right after class* (30–40 minutes, while the ideas are fresh) is worth more than three new problems tomorrow. On weekends, do the shared practice sets. For tricky, trick-based questions, repetition is the key.

1. Find the missing and repeated number

The story (how interviewers ask it): numbers $(1..n)$, all unique, are being transmitted. Due to interference, exactly one number got replaced by another number from the same range. At the destination you receive the array — one number is now **repeated**, so one number is **missing**. Find both.

original:	4	1	3	2	5	
received:	4	1	4	2	5	repeated = 4, missing = 3

We built a ladder of solutions:

1.1 Brute force — $O(n^2)$

For each value, scan the array to count it. Finds the repeated; then loop $(1..n)$ checking presence to find the missing.

1.2 Sort — $O(n \log n)$

After sorting, position i must hold $i+1$: $[1, 2, 4, 4, 5]$ — the first mismatch exposes everything. Downside: modifies the array.

1.3 Set / hash map — $O(n)$ time, $O(n)$ space

First value already in the set = repeated. Then check $(1..n)$ against the set for the missing one.

1.4 Negative marking — $O(n)$ time, $O(1)$ space (uses index as a hash map)

Values are $(1..n)$ and indices are $(1..n)$ (1-based thinking): **every value is also a position**. Visit value $v \rightarrow$ flip the sign at position v . Already negative? You've been here before $\rightarrow v$ is the repeated number.

```

received:  4   1   4   2   5
see 4  -> mark pos 4:   4   1   4  -2   5
see 1  -> mark pos 1:  -4   1   4  -2   5
see 4  -> pos 4 already negative => repeated = 4
see 2  -> mark pos 2:  -4  -1   4  -2   5
see 5  -> mark pos 5:  -4  -1   4  -2  -5
second scan: position 3 is still positive => missing = 3

```

(Remember to take `abs()` of what you read — it may already be marked.) This is an accepted, impressive interview solution. But the follow-up comes:

┆ "Can you do it **without modifying the array**?"

1.5 The math solution — $O(n)$ time, $O(1)$ space, array untouched

Two unknowns: missing **M**, repeated **R**. Two unknowns need two equations.

Equation 1. Sum what *should* be there, subtract what *is* there:

$$\begin{aligned}
 (a+b+c+d+e) - (a+b+c+c+d) &= e - c && \text{(everything else cancels)} \\
 &= M - R = X
 \end{aligned}$$

A tempting Equation 2 — and why it's bad. Multiply instead of add:

$(a \cdot b \cdot c \cdot d \cdot e) / (a \cdot b \cdot c \cdot c \cdot d) = e/c = M/R$. Mathematically fine, practically a disaster: products of n numbers **overflow** almost immediately. Even the famous formula $n(n+1)/2$ is risky for n around 10^8 in fixed-width integers. This is the same reason binary search writes

```
mid = lo + (hi - lo) / 2      // instead of (lo + hi) / 2
```

— we avoid building unnecessarily large intermediate values. You don't need the sum formulas at all: a plain loop that accumulates *differences* keeps every intermediate value small.

The good Equation 2. Do the same subtraction with **squares**:

$$\Sigma(\text{should-be})^2 - \Sigma(\text{received})^2 = M^2 - R^2 = Y$$

Now algebra you know: $M^2 - R^2 = (M+R)(M-R)$, so

$$\begin{aligned}
 (M + R) \cdot X &= Y && \Rightarrow && M + R &= Y / X \\
 M - R &= X \\
 \hline
 \text{add:} & \quad 2M &= X + Y/X && \Rightarrow & \quad M &= (X + Y/X) / 2 \\
 \text{subtract:} & \quad 2R &= Y/X - X && \Rightarrow & \quad R &= (Y/X - X) / 2
 \end{aligned}$$

The simpler you make the math before coding, the more beautiful the code:

```
python

def find_missing_and_repeated(arr):
    diff = sq_diff = 0                    # X = M - R, Y = M^2 - R^2
    for i, v in enumerate(arr):
        diff += (i + 1) - v              # should-be minus received
        sq_diff += (i + 1) * (i + 1) - v * v
    s = sq_diff // diff                  # s = Y / X = M + R
    missing = (diff + s) // 2            # (X + Y/X) / 2
    repeated = (s - diff) // 2           # (Y/X - X) / 2
    return missing, repeated

find_missing_and_repeated([4, 1, 4, 2, 5])    # -> (3, 4)
```

Check on the example: $X = 15 - 16 = -1$, $Y = 55 - 62 = -7$, $s = -7 / -1 = 7$, $M = (-1 + 7) / 2 = 3$, $R = (7 + 1) / 2 = 4$. ✓

(In Java/C++ use 64-bit integers for the accumulators — squares get big.)

2. Linked lists — the basics

Arrays need one **contiguous** block of memory, sized up front. Sometimes memory is fragmented — free space exists, but not in one piece. A **linked list** works with scattered memory: each node stores its value **and the address of the next node**. The chain replaces index arithmetic.

```
java

class Node<T> {
    T val;
    Node<T> next;    // null => this is the last node
}
```

	Array	Linked list
memory	one contiguous block	scattered nodes
access i-th element	$O(1)$ (address arithmetic)	$O(n)$ (walk the chain)
grow	may need full copy	just link a new node

Target for practice: at least **10 good linked-list problems** (start with "reverse a linked list").

3. Slow & fast pointers (tortoise and hare)

(Also called *Floyd's cycle detection*. Careful in interviews: **Floyd-Warshall** is a different algorithm — all-pairs shortest paths in graphs. Say "Floyd's cycle detection" or "tortoise and hare".)

The idea: two runners on a circular track with **different** speeds will meet. We use speeds 1 and 2:

python

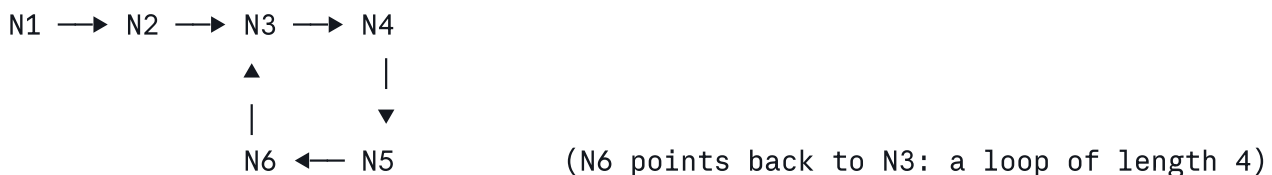
```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:           # they met inside a loop
            return True
    return False                  # fast fell off the end: no loop
```

If there is no loop, `fast` simply reaches the end (`None`) and we return False.

Common pitfall: both pointers start at `head`, so `slow == fast` is true *before the first step*. If you put `slow != fast` in the `while` condition, the loop never runs. Advance first, compare after — like the code above.

Interview depth question: "must the speeds be 1 and 2?" Two runners with different speeds on a circular track always cross — for *continuous* runners. But pointers **jump** node by node. With speeds 1 and 2 the gap between them changes by exactly 1 per step, so it must hit 0 — the fast pointer can never *jump over* the slow one. That guarantee (plus fewest operations) is why everyone uses 1 and 2.

Example



```
step 0:  slow = N1, fast = N1
step 1:  slow = N2, fast = N3
step 2:  slow = N3, fast = N5
step 3:  slow = N4, fast = N3
step 4:  slow = N5, fast = N5  < they meet at N5. Loop exists!
```

4. Finding the START of the loop

Follow-up question: not just "is there a loop", but **where does it begin?** (Answer for the picture above: N3.)

The algorithm (short enough to memorize, but understand the why):

1. Run slow/fast as above until they meet (say, at point **P**).
2. Reset ONE pointer to (head). Leave the other at P.
3. Move BOTH at speed 1. The node where they meet again is the loop start.

python

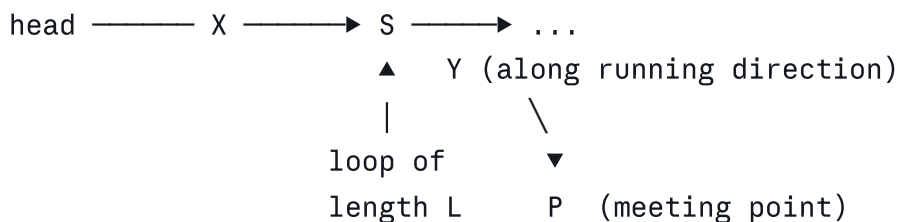
```
def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:                # phase 1: they met
            slow = head                  # phase 2: reset one pointer
            while slow is not fast:
                slow, fast = slow.next, fast.next
            return slow                  # the loop's first node
    return None                         # no loop
```

Watch it on the picture (phase 1 met at N5):

```
phase 2:  slow = N1, fast = N5
step 1:   slow = N2, fast = N6
step 2:   slow = N3, fast = N3    ← meet at N3 = loop start ✓
```

Why does this work? (the math interviewers probe)

Define three distances:



X = distance from head to S (the loop's start)
Y = distance from S to the meeting point P
L = loop length

By the time they meet:

$$\begin{aligned}\text{distance by slow} &= X + Y \\ \text{distance by fast} &= 2(X + Y) \quad (\text{double speed, same time})\end{aligned}$$

The fast pointer's *extra* distance was spent going around the loop — the difference between them must be some whole number of loops:

$$2(X + Y) - (X + Y) = X + Y = n \cdot L \quad \dots \text{for some whole number } n$$

Solve for X:

$$\begin{aligned}X &= n \cdot L - Y \\ &= (n-1) \cdot L + (L - Y)\end{aligned}$$

Read that equation like a story. **$L - Y$ is the distance from the meeting point P forward to the loop start S** (the rest of the lap). So it says:

Walking X steps from `(head)` lands you on S. Walking X steps from P = walking $(L - Y)$ to reach S, plus $(n-1)$ useless full laps — which also lands you on S. (Full laps are like distance without displacement: you moved, but you're back where you started.)

That is exactly phase 2: one pointer walks from `(head)`, one walks from P, both at speed 1 — after X steps **both stand on S**, so the first node where they meet is the loop start.

Verify with the picture: $X = 2$ ($N1 \rightarrow N3$), $L = 4$, meeting at $P = N5$ so $Y = 2$, $L - Y = 2$, $n = 1$. $X = 0 \cdot 4 + 2 = 2 \checkmark$ — and indeed phase 2 met after 2 steps.

Do this trace on 2-3 of your own examples with pen and paper. The proof needs nothing beyond this page, but *explaining* it smoothly needs practice — and interviewers do ask "why does resetting one pointer work?" even after you've written correct code.

(Preview: this same technique solves an array problem — "find the duplicate when you cannot modify the array and cannot use extra space" (LeetCode 287). Treating `(i → arr[i])` as a linked-list pointer, a duplicate value creates a loop. We'll connect these dots in an upcoming class.)

5. Stack — and the trap inside "implement a stack"

A stack is **LIFO** — last in, first out. Standard operations: `(push)`, `(pop)`, `(peek)` (look at top without removing), `(isEmpty)`, `(size)`.

Now the interview trap. "Implement a stack" has been asked at Amazon at senior levels, repeatedly — and people fail it *while writing working code*. Why? Because when an interviewer asks you to implement a **data structure**, they are not testing whether you can push and pop. They are testing whether you can **write a library** — this question is closer to LLD (low-level design) than to DSA.

So think like a library author:

1. Set the contract first — an interface, generic over the element type:

```
java

interface Stack<T> {
    T push(T ele);
    Optional<T> pop();           // Optional: popping an empty stack returns empty,
    Optional<T> peek();         // not an exception, not null
    int size();
    boolean isEmpty();
}
```

2. Then provide implementations against that contract:

```
java
```

```

class ArrayStack<T> implements Stack<T> {
    private Object[] data = new Object[10];    // start small
    private int top = -1;                      // index of the top element

    public T push(T ele) {
        if (top + 1 == data.length) grow();    // full -> double the capacity
        data[++top] = ele;
        return ele;
    }

    @SuppressWarnings("unchecked")
    public Optional<T> pop() {
        if (isEmpty()) return Optional.empty();
        T val = (T) data[top];
        data[top--] = null;                    // drop the reference (helps GC)
        return Optional.of(val);
    }

    @SuppressWarnings("unchecked")
    public Optional<T> peek() {
        return isEmpty() ? Optional.empty() : Optional.of((T) data[top]);
    }

    public int size() { return top + 1; }
    public boolean isEmpty() { return top == -1; }

    private void grow() { data = Arrays.copyOf(data, data.length * 2); }
}

class LinkedListStack<T> implements Stack<T> {
    private record Node<T>(T val, Node<T> next) {}
    private Node<T> head = null;
    private int n = 0;

    public T push(T ele) { head = new Node<>(ele, head); n++; return ele; }
    public Optional<T> pop() {
        if (head == null) return Optional.empty();
        T val = head.val(); head = head.next(); n--;
        return Optional.of(val);
    }

    public Optional<T> peek() { return head == null ? Optional.empty()
                                                         : Optional.of(head.val()); }

    public int size() { return n; }
    public boolean isEmpty() { return head == null; }
}

```

Talking points the interviewer is fishing for:

- **Generics** — a stack of anything, not `int[]` hard-coded.
- **Growth policy** — start with capacity 10; when full, double it (this is how real collections libraries behave; read Java's collections source!).
- **Edge behavior** — `pop` on empty returns `Optional.empty()`, a deliberate API decision you can defend.
- **Thread safety (common follow-up)**. You discuss, not necessarily code: `peek` has no side effects, so it needs no protection; only mutating operations do. You don't have to jump to heavyweight tools (synchronization, semaphores) — often a simpler device works, e.g. an atomic integer for the top index, or building on a non-blocking / concurrent structure internally. Interviewers want the *simple appropriate* solution, not the fanciest one.

Rule to remember: "implement a data structure" = "show me you can design a small library." Contract → implementations → policies (growth, emptiness, concurrency).

6. Monotonic stack

Setup problem: for each element, find the **first greater element to its right**.

arr:	3	1	4	5	2	6	4	8
answer:	4	4	5	6	6	8	8	None

Brute force: for each element scan rightward until something bigger — $O(n^2)$.

The observation that unlocks it: scan from **right to left** and keep candidates for "next greater". When a new element x arrives, every stored candidate **smaller than x becomes useless forever** — any element further left would prefer x over them. (Once you've seen a 6, a 4 behind it can never be anyone's "next greater".)

"Keep candidates, discard the ones a newcomer beats" is exactly what a **stack** does well — and because we only ever discard smaller ones, the stack stays sorted at all times.

Monotonic = always increasing (or always decreasing). Hence: *monotonic stack*.

Dry run (right to left; try `([3, 1, 4, 7, 2, 6, 4, 8])`)

element	pop (useless)	answer = new top	stack after push (top first)
8	-	None	8
4	-	8	4, 8
6	4	8	6, 8
2	-	6	2, 6, 8
7	2, 6	8	7, 8
4	-	7	4, 7, 8
1	-	4	1, 4, 7, 8
3	1	4	3, 4, 7, 8

Read any row: the stack is always sorted. Every element is pushed once and popped at most once → $O(n)$ total, even though there's a loop in a loop.

The template (memorize the shape, not the code)

```
for each element (order depends on the problem):
    while stack is not empty AND top is "useless" vs current:
        stack.pop()
    answer for current = stack.top (or None if empty)
    stack.push(current)
```

Concrete version for "next greater to the right":

```
python
def next_greater(arr):
    n = len(arr)
    ans = [None] * n
    stack = []
    for i in range(n - 1, -1, -1):          # right to left
        while stack and stack[-1] <= arr[i]:
            stack.pop()                     # remove useless elements
        ans[i] = stack[-1] if stack else None
        stack.append(arr[i])
    return ans
```

Four variations, one template

Problem	Scan direction	Pop while top is...
next greater to the right	right → left	<code>top <= current</code>
next smaller to the right	right → left	<code>top >= current</code>
previous greater (to the left)	left → right	<code>top <= current</code>
previous smaller (to the left)	left → right	<code>top >= current</code>

Only the direction and the "useless" condition change. Write this template five or six times and these problems become 2–3 minute problems — that is the speed to aim for.

Where it shows up

Famous problems that are this pattern wearing a story: **Trapping Rain Water** (a building holds water only if a taller building exists beyond it), **Largest Rectangle in a Histogram**, stock span, daily temperatures. The story is decoration; your job in an interview is to strip the story and expose the raw question ("for each element, find the next greater/smaller...") — fast. People who need ten minutes to extract the raw problem struggle regardless of coding skill.

Coming up

- The slow/fast pointer trick applied to an **array** (LeetCode 287 — find the duplicate with no modification and $O(1)$ space).
- **Queue** — the next data structure after stack.

Practice checklist for this class

1. Re-implement `find_missing_and_repeated` from the two equations without looking at the notes.
2. Trace `cycle_start` on a list you invent (different X, L, Y) and check $X = (n-1) \cdot L + (L-Y)$ holds on your numbers.
3. Ten linked-list problems, starting with "reverse a linked list".
4. Write the monotonic-stack template from memory for all four variations.
5. Then: Daily Temperatures (LC 739), Next Greater Element II (LC 503), Trapping Rain Water (LC 42).